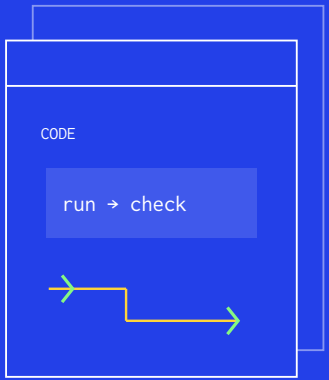
WEEK 3 / LECTURE 06

MLOps: Machine Learning Operations

Continuous Integration for ML Projects

How a change earns the right to merge: clean machines, honest checks, tests that can fail.



After this lecture you can

Each outcome ends in evidence you can show from your repository.

1. **Read a run**
   Say what a CI run proves, and what it cannot, from the commit it ran on.
2. **Design a pipeline**
   Order cheap checks first, give each job a reason, keep permissions minimal.
3. **Test for ML defects**
   Write regression tests that fail for named defects, and prove they can fail.

What did your Week 1–2 workflow really check?

Continuous integration only protects what it runs.

RAN

Setup and style

Preflight, Ruff, infra tests and the Lab 1 tests.

NEVER RAN

Most of the project

Exercise tests, API tests and the container build.

NO RUN

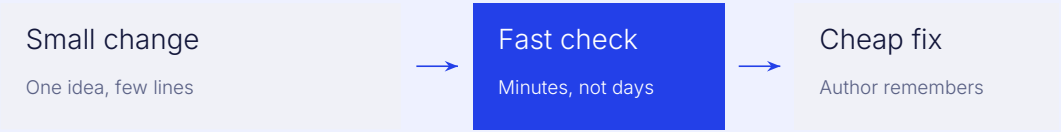
Some branches

Pull requests into develop did not start it.

Open your own ci.yml: which line decides each of these?

Why integrate continuously

Small changes, checked quickly, are cheap to fix.



`A defect found late costs more to find and to fix.`

Integration problems grow with the time between merges.

CI, CD and CT are three processes

Related, but not three steps of one mandatory pipeline.

CI

Integrate

Check every revision: build, test, quality.

CD

Deliver

Prepare a verified package; promotion stays controlled.

CT

Train

Repeat training with data checks and evaluation (Lecture 7).

Today: CI. Lectures 7 and 8 add training gates and controlled releases.

WHAT A GREEN RUN MEANS

Green means: this revision, this runner, these checks.

Not that the code is correct, and not that your laptop agrees.

A clean machine has none of your files

Know what the runner has before you trust a green run.

THE RUNNER HAS

What the commit gives it

- A checkout of the commit
- A fresh operating system image
- What your lock file installs

THE RUNNER LACKS

Everything else

- Your .venv and caches
- Ignored folders such as artifacts/ and tracking/
- Your secrets and credentials
- Any file that exists only on your laptop

If a check passes only because of a local file, CI will tell you.

Local and CI evidence are different claims

Say which one you are citing.

LOCAL

Your terminal

Fast and yours; can depend on anything on your machine.

CI

The runner

Slower and shared; built from the commit and the lock only.

RULE

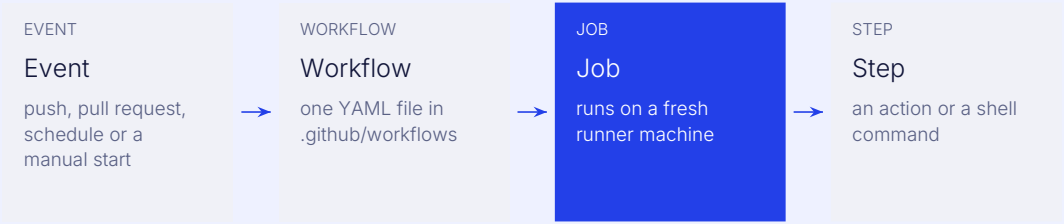
Never mix them

A result from one is not evidence for the other.

Lab 2 already drew this line for MLflow runs; CI keeps it.

A workflow is events, jobs and steps

Each job starts from nothing; jobs share only what you hand over.



`Nothing from your laptop is on the runner.`

Design question: what must each job be given?

Triggers decide what is protected

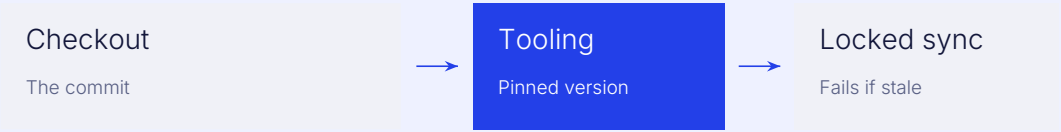
Ask the question in the last column for your own team.

Trigger	Starts when	Question to ask
pull_request	a PR targets a listed branch	Which branches does your team merge into?
push	a commit lands on a listed branch	Is the integration branch checked after a merge?
workflow_dispatch	someone starts it by hand	What do you want to rerun on demand?
schedule	a cron time arrives	What should be checked when nobody pushes?

Gitflow has two integration branches. A filter that lists only one leaves the other unchecked.

Install from the lock, on every run

The same rule as Lab 3, now enforced by a machine.



`A stale lock must stop the run.`

Never regenerate the lock to get green.

Jobs are independent unless you say otherwise

How one job hands work to the next is a design decision.

PARALLEL

By default

Jobs start together on separate machines.

ORDERED

With needs

A job waits for the jobs it needs, and is skipped if they fail.

HAND-OVER

Artifacts

Files travel between jobs only if you upload and download them.

Design question: what must the next job be given?

Least privilege, pinned actions, time limits

Three habits your starter workflow already has.

TOKEN

Minimal permissions

Give the workflow only what it needs; read-only is the safe default.

PIN

Pin what you run

A tag can move; a commit SHA cannot. Update pins deliberately.

LIMIT

Time-limit every job

A hung job should stop, not use up your minutes.

Keep all three when you extend the file.

Cost is a design input

Every run uses minutes, and private repositories draw on a monthly allowance.

ORDER

Cheap first

Style and fast tests before container builds.

CANCEL

Cancel stale runs

A newer push can cancel the older run of the same branch.

SCOPE

Run what matters

Not every change needs every check; say which does.

Fast feedback is also a cost: slow pipelines get ignored.

“No run appeared”: four places to look

Most missing runs are one of these.

Place	What to check
Location	The file is not in .github/workflows/ at the repository root
Trigger	The branch filter does not include the PR's target branch
Syntax	A YAML error: the Actions tab marks the workflow as invalid
Switch	Actions is disabled for the repository or for the workflow

Fix one at a time and read the Actions tab after each push.

Reading the Checks tab

Three levels: run, job, step. Start at the first red one.

RUN

One per event

A run belongs to one commit and one trigger.

JOB

One per machine

Open the failed job; the first red step is your lead.



STEP

One per command

Read the log above the error, not only the last line.

Save what you read as text; the page will not last.

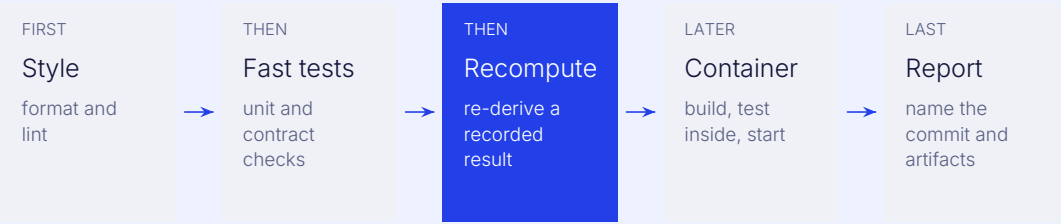
DESIGN RULE

If CI can run it,
you can run it locally.

Use the same locked command in the workflow and in your terminal.

Order the stages by cost and purpose

Fail fast: a typo should not cost a container build.



An example order from a spam-filter project; your project may need another.

Decide which checks block a merge

Block only on checks that are fast, stable and meaningful.

REQUIRED

Blocks the merge

- Fast and stable
- Fails only for a real defect
- Everyone agrees what red means

INFORMATIONAL

Reports, does not block

- Slow or noisy
- Exploratory or new
- Promote it once it is trusted

On free private repositories GitHub cannot block the merge button for you; your team agreement does.

A fresh checkout is empty of models

Ignored folders are not in Git, so the runner cannot see them.

FACT

Nothing local

artifacts/ and tracking/ are ignored by Git, so a fresh checkout has no model.

OPTIONS

Produce or stand in

Build it on the runner or use a small stand-in; each proves something different.

CLAIM

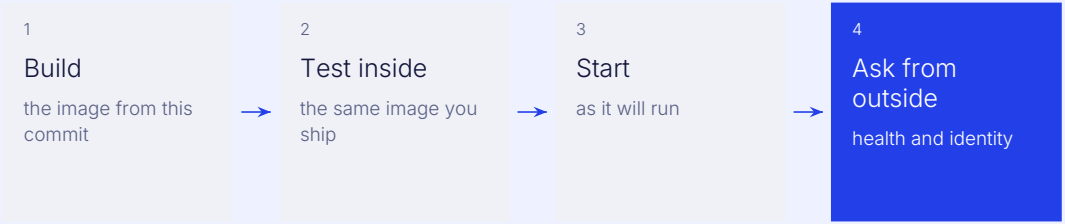
Claim only what ran

What the runner produced is CI evidence. Your laptop's runs are not.

Which option fits which claim is your decision.

Prove it where it will run

A container job checks the thing you would ship.



The run should print the commit and the image it checked.

Make the run say what it built

Evidence must name its subject.

REVISION

Commit SHA

Which code was checked.

IMAGE

Image ID

Which build was tested.

MODEL

Model version

Which model answered.

A job summary or a log line is enough; a screenshot alone does not connect them.

What comes after CI

Each lecture adds one kind of control.

NOW

CI

Every change is checked on a clean machine.

LECTURE 7

Training gates

A retrained model must earn promotion.

LECTURE 8

Incident response

A bad release can be noticed and reversed.

CI is the foundation: later controls assume trustworthy checks.

A regression test protects what already works

It exists so that a future change cannot quietly undo it.

1 **Pin a behaviour**

Choose something that worked and must keep working.

2 **Name the defect**

State which mistake would make the test fail.

3 **Keep it stable**

Same input, same answer: no hidden dependence on time, order or luck.

Kinds of regression tests for ML systems

Examples come from a spam filter; yours will differ.

Kind	Catches	Spam-filter example
Contract	the interface changes	empty text returns an error, not a score
Data	data or splits change silently	file hash and row counts still match
Recompute	training changes silently	metric within a justified tolerance of the recorded one
Integrity	files and metadata disagree	the stored hash equals the file's hash
Invariant	the model is nonsense	a score is always between 0 and 1

A bug-fix test is a regression test written on the day you fix the bug.

What belongs in CI, and what does not

CI is for fast, repeatable, meaningful checks.

BELONGS

Fast, stable, meaningful

- Style and unit tests
- Contract and data checks
- A small, repeatable recompute

DOES NOT

Slow, noisy, exploratory

- Hyperparameter search
- Long training runs
- Anything that depends on timing or luck

Heavy work can run on demand or on a schedule instead.

A recomputed result needs a tolerance

Numbers from two machines are rarely identical to the last digit.

WHY

Floating point differs

Libraries, CPUs and thread counts can change the last digits.

HOW

Measure, then decide

Repeat the run; the spread you see is your evidence for a tolerance.

NEVER

Tune it to pass

A tolerance chosen to turn the run green proves nothing.

Changing a tolerance is a reviewed change with a reason.

Flaky tests are defects too

A test that fails at random teaches the team to ignore red.

COMMON CAUSES

Where randomness hides

- Time and time zones
- Test order and shared state
- Random seeds and threads
- Network calls

WHAT TO DO

Remove the cause

- Fix seeds; isolate state
- Quarantine only with an owner and a date
- Never retry until green

A flaky check is worse than no check: it hides real failures.

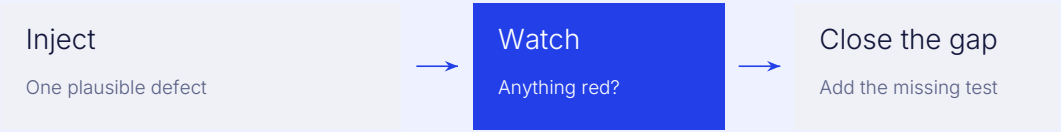
TESTS THAT CANNOT FAIL

A test that cannot fail proves nothing.

Ask of every test: which defect would turn this red?

Break it on purpose to learn what is covered

Mutation thinking: change the code and see who notices.



`If everything stays green, the pipeline has a gap.`

Do this on a throwaway branch or in a pull request you do not merge.

Good and bad reasons to change a test

A test is a statement about intended behaviour.

GOOD

The contract changed

Update the test and say why in the pull request.

BAD

To get green

Loosening an assertion hides the signal you built.

BEST

Fix the cause

If the test describes the right behaviour, change the code.

Every change to a test deserves the same review as a change to code.

Read a failure

Work from the first red step upwards.




```
Do not rerun until it passes: that hides flakiness.
```

Write the cause in your own words before you touch the code.

Reproduce with the same command

If you can run it locally, you can debug it.

ILLUSTRATIVE EXAMPLE

Copy the failing step's command from the log and run it from a clean environment, as the runner would.

terminal

```
$ uv sync --locked
$ uv run --locked pytest -q -x
```

Passes locally, fails in CI? The difference is the finding: files, versions, order.

Repair, do not mute

Each shortcut removes the signal the pipeline exists to give.

Tempting	Why it fails	Instead
continue-on-error, || true	hides the red	fix the failing step
skip or delete the test	removes the evidence	fix the code or justify
loosen the assertion	the test stops meaning anything	keep the contract
retry until green	hides flakiness	find the cause
run only the passing checks	unchecked code merges	run what matters

If you must change a check, change it in a pull request and say why.

Revert or fix forward?

Both are fine; the history should show which one you chose and why.

REVERT

Undo the change

- The change is large or unclear
- The branch is blocked for others
- You need green now

FIX FORWARD

Repair on top

- The cause is understood
- The fix is small
- The history stays readable

Either way the red run and its repair stay in the history.

THE LIMIT OF CI

Green is evidence, not proof.

CI shows that these checks passed on this commit. Whether they were the right checks is still your job.

Evidence that survives

Run pages are temporary; text you saved is not.

RECORDS

Expire

Actions keeps runs and logs for a limited time (90 days by default).

TEXT

Lasts

Save the failing log as text in your repository or report.

LINKS

Support

A link points to the evidence; the saved log is the evidence.

Check your repository's retention setting if you need longer.

Three ideas worth keeping

If you remember only these, you can rebuild the rest.

1 **Green has a scope**

This revision, this runner, these checks: nothing more is proved.

2 **A test is worth what it can catch**

Name the defect, then show that the test can fail.

3 **Evidence must survive**

Save the text, name the commit, say who produced it.

Where the ideas come from

Public documentation and well-known practice; read the originals for every option.

REFERENCES

Examples on slides are illustrative. Run them in your own project before you rely on them.



Check each source's current version before relying on it.

OPTIONAL APPENDIX

Go deeper, when you need it.

These slides extend the lecture. None is needed for the core ideas.

Matrix builds

Run the same job across several versions or systems.

WHAT

One job, many settings

Run the same job across several versions or systems.

WHY

Platform differences

A green run on one system says little about another.

COST

Minutes multiply

Use it where a platform difference is a real risk.

Caching dependencies

Reuse downloads between runs of the same lock.

WHAT

Reuse downloads

Keep installed packages between runs of the same lock.

KEY

The lock is the key

Change the lock and the cache must miss.

RISK

Stale caches

A cache can hide a broken install; clear it when in doubt.

Keeping pins up to date

A pin protects you until it falls behind.

PIN

Pins age

A pinned action or dependency stops receiving fixes.

BOT

Update bots

A bot can open a PR per update; your checks run on it.

REVIEW

You still review

A passing update is a candidate, not an approval.

Branch protection and rulesets

Where your plan allows it, GitHub can require checks before a merge.

WHERE AVAILABLE

GitHub can block the merge

- Required status checks
- Required reviews
- No direct pushes to main

FREE PRIVATE REPOSITORIES

Your agreement does it

- The setting is not available
- Write the rule in CONTRIBUTING.md
- Reviewers check the current commit

Public repositories on a free plan can use protected branches; check your own plan.

Routing reviews

Make the right person see the right change.

OWNERS

Owners per path

A file names who must review changes to which paths.

TEMPLATE

PR template

A checklist reminds authors to attach evidence.

HABIT

Rotate

Rotate reviewers so knowledge does not sit with one person.

Pre-commit: the same checks, earlier

A hook catches the easy problems before they reach CI.

LOCAL

Before the commit

Run style checks automatically when you commit.

CI

Again in CI

CI repeats them, because a hook can be skipped.

SAME

One definition

Use the same configuration in both places.

Schedules and manual runs

Not every check starts with a push.

SCHEDULE

On a timer

Check things that change without a push, such as dependencies.

MANUAL

On demand

Start a heavy or rare job by hand, with inputs.

COST

Mind the minutes

Scheduled runs use minutes whether or not anything changed.

Artifacts and retention

Files a run produces, and how long they last.

ARTIFACT

A file from a run

Upload results so a person or a later job can use them.

LIMIT

Retention is limited

Artifacts and logs expire after the configured period.

PLAN

Decide what to keep

Copy long-lived evidence into the repository or report.

Environments as stages of confidence

A change is promoted through places that look more and more like production.

SAME CODE

Moves forward

- Development: change freely
- Staging: rehearse the release
- Production: users depend on it

DIFFERENT SETTINGS

Stay outside the code

- Configuration differs per place
- Secrets differ per place
- Data differs per place

This course has local runs, CI runs and the final release; the idea is the same.

Infrastructure as code

Describe the environment in files, not in clicks.

IDEA

Describe, do not click

Write infrastructure as files that can be reviewed and versioned.

TOOLS

Many tools

Terraform is one example; this course uses none.

HABITS

The same habits

Review, test and roll back infrastructure like code.

Online testing: A/B, canary, shadow

These reduce the risk of a release; Lecture 8 returns to them.

A/B

Two versions

Two versions serve different users; compare outcomes.

CANARY

A small share first

A small share of traffic goes to the new version first.

SHADOW

Seen, not used

The new version sees real requests but its answers are not used.